

# Symbol-Equivariant Recurrent Reasoning Models

Richard Freinschlag<sup>1</sup> Timo Bertram<sup>1</sup> Erich Kobler<sup>1</sup> Andreas Mayr<sup>1</sup> Günter Klambauer<sup>1,2</sup>

## Abstract

Reasoning problems such as Sudoku and ARC-AGI remain challenging for neural networks. The structured problem solving architecture family of Recurrent Reasoning Models (RRMs), including Hierarchical Reasoning Model (HRM) and Tiny Recursive Model (TRM), offer a compact alternative to large language models, but currently handle symbol symmetries only implicitly via costly data augmentation. We introduce *Symbol-Equivariant Recurrent Reasoning Models* (SE-RRMs), which enforce permutation equivariance at the architectural level through symbol-equivariant layers, guaranteeing identical solutions under symbol or color permutations. SE-RRMs outperform prior RRM on  $9 \times 9$  Sudoku and generalize from just training on  $9 \times 9$  to smaller  $4 \times 4$  and larger  $16 \times 16$  and  $25 \times 25$  instances, to which existing RRM cannot extrapolate. On ARC-AGI-1 and ARC-AGI-2, SE-RRMs achieve competitive performance with substantially less data augmentation and only 2 million parameters, demonstrating that explicitly encoding symmetry improves the robustness and scalability of neural reasoning. Code is available at <https://github.com/ml-jku/SE-RRM>.

## 1. Introduction

**The difficulty of reasoning problems.** Despite their huge successes, deep neural networks and large language models (LLMs) have difficulties at solving reasoning problems (Hazra et al., 2024; Malek et al., 2025; Seely et al., 2025), such as undergraduate-level physics (Xu et al., 2025), chemical structures (Mirza et al., 2025; Bartmann et al., 2026), or real-world clinical scenarios (Kim et al., 2025). Although much of the research community has focused on abstract

<sup>1</sup>ELLIS Unit Linz, LIT AI Lab & Institute for Machine Learning, Johannes Kepler University Linz, Austria <sup>2</sup>Clinical Research Institute for Medical AI, Johannes Kepler University Linz, Austria. Correspondence to: Günter Klambauer <lastname@ml.jku.at>.

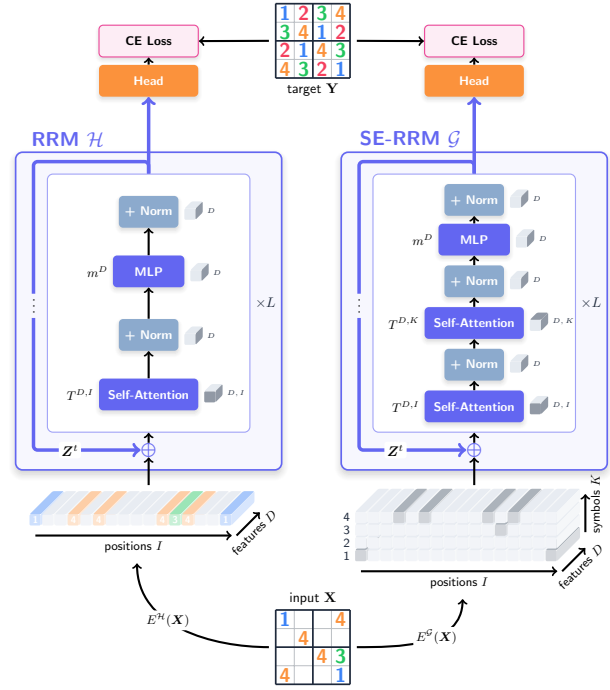


Figure 1. Comparison of a RRM, in particular TRM (Jolicoeur-Martineau, 2025), (left) and the proposed SE-RRM (right) for solving a  $4 \times 4$  Sudoku. For RRM, the puzzle is encoded with a symbol-specific embedding at every position, resulting in a 2D matrix. In contrast, SE-RRM introduces a third dimension to link positions and symbols and uses the same embedding for all symbols. SE-RRM is symbol-equivariant, since both Self-Attention  $T^{D,I}$  and  $T^{D,K}$  layers in the transformer block operate subsequently in position and symbol dimension, and the norm, addition, and MLP layers are applied token-wise.

benchmarks such as ARC-AGI tasks (Chollet, 2019; Chollet et al., 2025), Sudoku (Seely et al., 2025; Long, 2023), and maze problems (Wang et al., 2025), these settings can serve as proxies for real-world reasoning challenges, including planning, diagnosis, legal interpretation, causal analysis, social interaction, and risk assessment. While such benchmarks do not capture the full richness of uncertainty, social interaction, or normative ambiguity, they isolate the constraint-satisfaction and combinatorial structure central to many practical problems (Knuth, 2015). In particular, Sudoku offers a clean testbed for evaluating neural networks reasoning capacities under hard constraints and structural

symmetry.

**Symbolic solvers** Structured problems such as combinatorial optimization and constraint satisfaction have traditionally been solved using symbolic methods such as SAT, constraint programming (CP), and mixed-integer optimization (MIP) solvers (Biere et al., 2021; Zhang et al., 2023; Bengio et al., 2021; Kotary et al., 2021). These approaches rely on explicit formulations and combinatorial search, and can provide strong correctness and completeness guarantees; backtracking-based Sudoku solvers find a solution when one exists and certify infeasibility otherwise. However, symbolic solvers can be computationally expensive: solving generalized  $n^2 \times n^2$  Sudoku is NP-hard (Yato & Seta, 2003) with worst-case complexity  $O((n^2)^{n^4})$ . Learning-based methods can accelerate search by learning solver policies (Bengio et al., 2021), while large language models have also been explored, but without task-specific scaffolding their performance on tightly constrained problems remains limited (Hazra et al., 2024; Malek et al., 2025; Seely et al., 2025).

**LLM-based solutions for reasoning problems.** Large language models (LLMs) (Peters et al., 2018; Devlin et al., 2019; Brown et al., 2020) show limited ability to solve reasoning problems expressed in natural language (Wei et al., 2022b;a; Srivastava et al., 2023), though this can be improved through reinforcement learning and large-scale post-training (Guo et al., 2025). Their reasoning is typically evaluated on text-based benchmarks such as BIG-Bench (Srivastava et al., 2023), which are far simpler than structured combinatorial, constraint satisfaction, or SAT problems. On harder symbolic tasks, the performance of plain LLM approaches degrade sharply: frontier models struggle on 3-SAT (Hazra et al., 2024) and achieve below 5% on ARC-AGI-2 (Chollet et al., 2025). However, aside from this strategy to use LLMs to directly predict the output from the input (Cole & Osman, 2025), LLMs can also be used to infer latent functions, such as Python programs (Li et al., 2024; Ellis et al., 2020; Pourcel et al., 2025; Mirchandani et al., 2023). The latter currently appears more promising, since the strongest ARC-AGI-2 results rely on heavily orchestrated, LLM-guided symbolic pipelines with extensive human-designed components and high computational cost (Poetik, 2025). As a result, despite their strong performance, LLM-based approaches remain a limited solution for general reasoning.

**Recurrent Reasoning Models (RRMs) for structured problems.** Recent work has introduced Recurrent Reasoning Model architectures, notably Hierarchical Reasoning Model (HRM) (Wang et al., 2025) and Tiny Recursive Model (TRM) (Jolicoeur-Martineau, 2025), to address discrete reasoning benchmarks such as Sudoku, Maze, and ARC-AGI. RRM are a class of neural architectures de-

signed to perform structured, multi-step problem solving through iterative refinement. At their core, RRM consist of a neural computation block that is applied repeatedly in a fixed-point iteration scheme. Rather than producing a solution in a single forward pass, the model repeatedly updates its recurrent state, allowing outputs to be produced at intermediate steps. Each block operates on two primary inputs: (i) an embedding of the structured problem instance, and (ii) a recurrent state that maintains a representation of the computation from prior iterations. The block produces an updated recurrent state, from which task-specific outputs can be derived or decoded at any iteration step. Training applies a fixed supervision signal, representing the task solution, repeatedly throughout the fixed-point iteration. After each application of the supervision signal, gradients are detached. Then several further iterations of the recurrent block without supervision to refine the recurrent state are performed, after which the same supervision signal is applied again to the most recent iteration step(s). Model parameters are optimized by applying this procedure across a dataset of structured problem-solving tasks. The iterative updates can be combined with a learned stopping criterion, enabling adaptive computation time and dynamic control over the number of reasoning steps per task instance.

**Constant-input RNNs.** The use of constant inputs in RRM is not without precedent, as constant “plan” vectors were already employed in early Jordan-type recurrent networks (Jordan, 1986) to study internally generated temporal dynamics. While Deep Equilibrium Models (DEQs) (Bai et al., 2019) similarly rely on fixed external inputs, they depart from recurrent formulations by computing equilibria through fixed-point solvers and optimizing them via implicit differentiation instead of time-unrolled dynamics. As noted in the DEQ literature, this line of work traces back to early equilibrium recurrent networks trained via recurrent backpropagation (Almeida, 1987; Pineda, 1987; Schmidhuber, 2015). Adaptive computation, particularly at test time, serves as a natural application setting for DEQs and is likewise a key component of RRM.

**Advantages of RRM.** RRM represent a promising direction for structured reasoning and complement LLM-based methods. As a type of RNN, they can natively perform an arbitrary number of sequential computations, which is required for state tracking and solving many types of reasoning problems (Merrill et al., 2025). If LLMs struggle to reason effectively, their robustness to distributional shift may be limited (Kim et al., 2025). Given that RRM are trained with an explicit focus on reasoning tasks, it is plausible that they could offer improved robustness under distributional shift. These characteristics are particularly relevant in domains such as combinatorial optimization and scheduling, where problem sizes and structures vary widely. Although RRM may introduce higher inference latency due to recurrence,

they are often parameter-efficient and can be trained with limited data, without large-scale pretraining or reinforcement learning. Moreover, RRM decouple reasoning depth from model capacity, enabling compute to scale flexibly at inference time. Some of these advantages have already shown practical promise, for example in insurance pricing with TRMs (Padayachy et al., 2026).

**Limitations of RRM.** A key limitation of existing recurrent reasoning models (RRMs) is the lack of explicit symbol equivariance. In many reasoning problems, such as Sudoku, ARC-AGI, and combinatorial optimization, symbols are interchangeable and solutions should be equivariant under symbol permutations (Cohen & Welling, 2016; Bronstein et al., 2017). Current RRM do not encode this symmetry architecturally and instead rely on costly data augmentation, which increases sample complexity and can hinder generalization to unseen symbol configurations. As a result, RRM may inefficiently learn redundant distinctions between equivalent symbols, limiting both training efficiency and robustness.

**Contributions** **a)** We introduce *Symbol-Equivariant Recurrent Reasoning Models (SE-RRMs)*, a new class of recurrent reasoning architectures that are explicitly equivariant under permutations of equivalent symbols, such as digits in Sudoku or colors in ARC-AGI problems.

**b)** We demonstrate that SE-RRMs reduce reliance on heavy data augmentation. In cases where symbol equivariance is not tackled with extensive data augmentation, SE-RRMs perform substantially better than other RRM, with only a fraction of trainable parameters. **c)** We show that symbol equivariance at the architectural level enables the integration of new symbols during inference, while preserving or improving the test-time scaling properties of recurrent reasoning.

**Related work.** Permutation equivariance has previously been explored in deep sets (Zaheer et al., 2017) and graph neural networks (Scarselli et al., 2008; Kipf & Welling, 2017; Defferrard et al., 2016; Gilmer et al., 2017) that are typically invariant or equivariant with respect to permutations of nodes of a graph. In reinforcement learning, generalization has been improved by augmenting action and state embeddings with random projections (Kirsch et al., 2023) or even by using completely random action embeddings (Sinii et al., 2024). In the latter case, the model is equivariant with respect to the permutation of actions. Moreover, permutation equivariance with respect to positions is a key property of attention operations as introduced by the transformer architecture (Vaswani et al., 2017). The transformer architecture has already been applied to problems with multiple dimensions. For example, Axial-Attention (Ho et al., 2019) applies one transformer block along the rows and

another transformer block along the columns of an image, while Criss-Cross Attention (Huang et al., 2019) applies the same principle in the domain of image segmentation. The MSA-Transformer (Rao et al., 2021) employs an attention operation in sequence and another attention operation in residue direction for multiple-sequence alignments of different protein sequences. SE-RRM realizes a permutation equivariant recurrent layer via axial-attention, specifically, it applies a self-attention layer across different symbols of the problem.

## 2. Symbol-equivariant Recurrent Reasoning Models (SE-RRMs)

### 2.1. Preliminaries: Vanilla RRM

**Goal.** The aim is to learn a machine learning model for structured problem-solving tasks, such as Sudoku, Maze, or ARC-AGI. We assume access to a training dataset containing representations of tasks together with their corresponding solutions, as well as a separate evaluation dataset. The datasets may consist of tasks that all follow the same set of rules, as in Sudoku, or of tasks governed by different underlying rules, as in ARC-AGI, where multiple task types are present. Depending on the complexity of a task type, multiple task instances are typically required to infer its governing rules. For datasets involving multiple task types (e.g. ARC-AGI), typically a task-type identifier is used to distinguish between different types.

Each task  $\mathbf{X}$  is represented as a tuple of symbols  $x_i \in \mathcal{C}$ , i.e.,  $\mathbf{X} = (x_1, \dots, x_I)$ . The task is assumed to be defined over an underlying structured domain, which is flattened into a fixed ordering to obtain the tuple representation. Each symbol  $x_i$  denotes the value associated with the  $i$ -th element of this flattened domain. The set  $\mathcal{C}$  consists of  $K$  symbol types ( $K = |\mathcal{C}|$ ) of the problem-solving task (e.g., the digits of a Sudoku together with a mask symbol) and is often also referred to as the set of colors. The solution  $\mathbf{Y} = (y_1, \dots, y_I)$  of a task is represented analogously as a tuple of symbols  $y_i \in \mathcal{C}$ , sharing the same indexing and flattened domain as  $\mathbf{X}$ . Depending on the specific rules of a task, some symbols in  $\mathcal{C}$  may be absent from the task description, the solution, or both (e.g., the mask symbol in Sudoku appears only in the input). In a dataset of tasks, a task may additionally be enriched with a task-type identifier  $p \in \mathbb{N}$ . In that way, there is an indication, which tasks follow the same rules and the set of training tasks, then consists of pairs  $(\mathbf{X}, p)$  instead of plain tasks  $\mathbf{X}$ .

**RRM blocks.** We abstract the principal neural computation block of RRM, as described above, as a mapping  $\mathcal{H} : \mathbb{R}^{D \times I} \times \mathbb{R}^{D \times I} \rightarrow \mathbb{R}^{D \times I}$ . We assume the recurrent state at time step  $t$  is denoted by  $\mathbf{Z}^t \in \mathbb{R}^{D \times I}$ . An embedding of the structured problem-solving task is provided by

$E^{\mathcal{H}}(\mathbf{X})$ . The RRM block then computes the subsequent recurrent state  $\mathbf{Z}^{t+1}$  from the task embedding  $E^{\mathcal{H}}(\mathbf{X})$  and the previous recurrent state  $\mathbf{Z}^t$ :

$$\mathbf{Z}^{t+1} = \mathcal{H}(E^{\mathcal{H}}(\mathbf{X}), \mathbf{Z}^t), \quad (1)$$

**Task Embedding.** An embedding  $E^C : C^I \rightarrow \mathbb{R}^{D \times I}$ , which maps a symbolic task description into a real vector space, is required to process a task. To construct such an embedding, approaches such as HRM or TRM employ a mapping  $C \rightarrow \mathbb{R}^D$  that assigns each symbol within a task alphabet to a  $D$ -dimensional real vector. This symbol-level mapping is applied uniformly across all positions in the task description (element-wise in the tuple). The resulting matrix  $\mathbf{E} = E^C(\mathbf{X})$  can be interpreted, in the context of attention-based processing, as a sequence of tokens  $e_i \in \mathbb{R}^D$ , i.e.,  $\mathbf{E} = (e_1, \dots, e_I)$ . Since the embedding  $\mathbf{E}$  represents a sequence of symbols originating from a structured domain, it is often beneficial to incorporate positional information. Positional embeddings (e.g., RoPE, [Su et al. \(2021\)](#)), can be added to the tokens  $e_i$  to enable attention-based operations to reason about the relative positions and relationships of symbols within the domain. Formally, let  $E^P : C^I \rightarrow \mathbb{R}^{D \times I}$  denote a positional embedding mapping that assigns a  $D$ -dimensional vector to each position  $i = 1, \dots, I$  in the original tuple  $\mathbf{X}$ . The combined task embedding  $E^{\mathcal{H}}$  can then be defined as

$$\begin{aligned} E^{\mathcal{H}} : C^I &\rightarrow \mathbb{R}^{D \times I}, \\ E^{\mathcal{H}} &:= E^C + E^P, \end{aligned} \quad (2)$$

where the symbol-level embedding  $E^C(\mathbf{X})$  of a tuple  $\mathbf{X}$  captures the content of the symbols in  $\mathbf{X}$  and the positional embedding  $E^P(\mathbf{X})$  encodes the respective positions.

When a dataset comprises multiple different task types, the embedding function  $E^{\mathcal{H}}$  is extended to encode task-type information and therefore becomes a function of two arguments, namely the task instance  $\mathbf{X}$  and its associated task-type  $p$ . In principle, this embedding can be realized in different ways. In HRM and TRM, the task type is represented by additional positions  $P$  whose embedding corresponds to a learned mapping  $\mathbb{N} \rightarrow \mathbb{R}^{D \times P}$ , which embeds the task type  $p$  as a matrix  $\in \mathbb{R}^{D \times P}$ . The overall task embedding is then obtained by augmenting the original task representation with this task-type embedding, resulting in

$$E^{\mathcal{H}} : C^I \times \mathbb{N} \rightarrow \mathbb{R}^{D \times (I+P)}. \quad (3)$$

**Solution Prediction.** The overall goal is to predict the correct solution of a task, i.e., to predict the correct tuple of symbols  $y_i \in C$  representing the solution. Since this defines a discrete correct/incorrect loss, RRM employs a differentiable surrogate loss for each position in the solution tuple. RRM typically output predicted probabilities for

each symbol at every position in the tuple. A categorical cross-entropy loss can then be applied independently to each position.

To obtain the predicted probabilities for the symbols, RRM typically map the final recurrent state  $\mathbf{Z}^T \in \mathbb{R}^{D \times I}$  at iteration step  $T$  to a logit prediction  $\hat{\mathbf{Y}} \in \mathbb{R}^{K \times I}$ . A softmax activation function, or a similar function such as stablemax, is applied to these logits, allowing the categorical cross-entropy loss with respect to the true training solution symbols to be computed. For the output mapping  $\mathbf{O} : \mathbb{R}^{D \times I} \rightarrow \mathbb{R}^{K \times I}$  from the final recurrent state to the logits, a single function  $\mathbb{R}^D \rightarrow \mathbb{R}^K$  may be applied independently along the position dimension of the recurrent state (i.e., shared across all positions). Usually, this is realized by just using a simple linear mapping (parameterized by a matrix  $\in \mathbb{R}^{D \times K}$ ).

**Overall RRM Architecture & Deep Supervision.** The core of the RRM architecture is the application of the subsequent fixed-point operation described in Equation (1), where  $\mathbf{Z}^0$  is fixed but randomly drawn and is constant with respect to the position index. Variations of this principle exist: both HRM and TRM can be seen as adaptations of RRM, as they structure multiple consecutive iteration steps of the block to superblocks. Consecutive blocks often omit the task embedding and add recent previous recurrent states to the embedding input, which allows some fixed-point iteration steps to incorporate both the immediately preceding and a more recent earlier state.

An important technique employed in both HRM and TRM is deep supervision. Instead of backpropagating the error from the final prediction through the entire fixed-point iteration scheme, prediction outputs are computed from selected intermediate recurrent states, i.e.,  $\mathbf{O}(\mathbf{Z}^t)$ , and the model parameters are updated immediately based on the gradient of the corresponding loss at time  $t$ :  $L(\mathbf{Y}^t, \mathbf{O}(\mathbf{Z}^t))$ . Gradients are computed only with respect to the last iteration step(s) immediately preceding the recurrent state, from which the output is computed.

**Neural Architecture of RRM blocks.** An RRM block  $\mathcal{H}$  realizing one step of the fixed-point iteration, consists of  $L$  stacked Transformer-style layers that process the task embedding  $E^{\mathcal{H}}(\mathbf{X})$  together with the recurrent state  $\mathbf{Z}^t$ , as shown in Equation (1). At certain time steps, variants of the architecture may additionally incorporate a recent previous recurrent state  $\mathbf{Z}^{t-r}$  or omit the task embedding, which can be expressed as

$$\mathbf{Z}^{t+1} = \mathcal{H}(\mathbf{1}_{\text{emb}} E^{\mathcal{H}}(\mathbf{X}) + \mathbf{1}_{\text{prev}} \mathbf{Z}^{t-r}, \mathbf{Z}^t), \quad (4)$$

where  $\mathbf{1}_{\text{emb}}$  and  $\mathbf{1}_{\text{prev}}$  are indicators equal to 1 when the embedding or previous state are included, and 0 when they are omitted. The exact wiring of plain  $\mathcal{H}$  blocks to superblocks in variants of RRM does not appear to be critical, as some

works (ARC Prize Team, 2025; Liao & Poggio, 2026) suggest that it may be replaceable by a stack of transformer layers (with shortcut access to previous layers).

The first layer of an RRM block, which we refer to as the initial layer  $H_0$ , is defined as the sum of all included inputs, i.e.,

$$H_0 := \mathbf{1}_{\text{emb}} E^{\mathcal{H}}(X) + \mathbf{1}_{\text{prev}} Z^{t-r} + Z^t. \quad (5)$$

This initial layer is then processed using the following update scheme

$$H'_l = \text{Norm}(H_l + T^{D,I}(H_l)), \quad \underline{D} \times \underline{I} \quad (6)$$

$$H_{l+1} = \text{Norm}(H'_l + m^D(H'_l)) \quad \underline{D} \times I. \quad (7)$$

for  $L$  additional steps, producing the block output  $Z^{t+1} = H_L$ . Each RRM block primarily consists of two types of operations. The first are self-attention layers (Vaswani et al., 2017), denoted as  $T^{D,I} : \mathbb{R}^{D \times I} \rightarrow \mathbb{R}^{D \times I}$ , which operate along the position dimension of the  $D \times I$  matrices. The second are position-wise MLPs, specifically SwiGLU modules (Shazeer, 2020), denoted  $m^D : \mathbb{R}^D \rightarrow \mathbb{R}^D$ , which are applied independently to each of the  $I$  positions. The utilized Norm operation is a root mean square normalization (Zhang & Sennrich, 2019) across the  $D$  features, which is also applied independently to all  $I$  positions.

## 2.2. Notable Properties of RRM

The structural design of RRM endows them with certain properties that can be formally characterized and leveraged in extensions. Among these properties, we note an interesting relationship to positional equivariance: without positional embeddings, RRM would be equivariant to any permutation applied to the input positions. The following proposition makes this more precise.

**Proposition 2.1.** *An RRM block  $\mathcal{H}$  without positional embeddings, as defined in Equations (5) to (7) is equivariant under permutations  $\pi : [I] \rightarrow [I]$  of the input positions  $\in [I] := \{1, \dots, I\}$ , i.e.,*

$$\begin{aligned} & \Pi_2^\pi \left( \mathcal{H} \left( \mathbf{1}_{\text{emb}} E^{\mathcal{H}}(X) + \mathbf{1}_{\text{prev}} Z^{t-r}, Z^t \right) \right) = \\ & = \mathcal{H} \left( \mathbf{1}_{\text{emb}} E^{\mathcal{H}}(\Pi_2^\pi(X)) + \mathbf{1}_{\text{prev}} \Pi_2^\pi(Z^{t-r}), \Pi_2^\pi(Z^t) \right), \end{aligned}$$

where

$$\begin{aligned} \Pi_a^\pi : \mathcal{S}^{d_1 \times \dots \times d_N} & \rightarrow \mathcal{S}^{d_1 \times \dots \times d_N} \\ (\Pi_a^\pi(A))_{i_1, \dots, i_N} & := A_{i_1, \dots, i_{a-1}, \pi(i_a), i_{a+1}, \dots, i_N}. \end{aligned}$$

The proposition is straightforward to show. It relies on the equivariance property of self-attention (Ilse et al., 2018; Lee et al., 2019). Further, since MLPs and the Norm operation

are applied independently to each position, they naturally fulfill equivariance wrt. position permutations. A similar argument holds for the residual additions. Since the pure symbol embedding  $E^C$  is also applied independently to each position, it can be shown that a whole RRM block is position-equivariant by induction. We can even further extend this equivariance property to a whole RRM architecture without the use of any positional embeddings, considering that  $Z^0$  is constant with respect to the position index and assuming that  $O$  is constructed from a single function being independently applied along the position dimension.

**Position-equivariance** While equivariance is often a desired property, this is not true for strict position-equivariance. Identical tokens in different positions can carry vastly different implications for the correct result, as observed in Sudoku, Maze, and ARC-AGI.

**Extrapolation beyond training cardinalities (positions and/or symbols).** Although trained with a fixed number of positions  $I$ , the architecture can operate at test time on inputs with a different number of positions  $J \neq I$ , and some degree of generalization might be expected (Kazemnejad et al., 2023; Anil et al., 2022). By contrast, it does not extrapolate to unseen symbols: problems with a larger symbol/color set  $|C'| > |C|$  with  $C \subset C'$  cannot be handled, as vanilla RRM encode symbols by mapping them via  $E^C$  to a high-dimensional feature space. Extrapolation to unseen symbols may specifically be relevant in few-shot settings if several new symbols appear at test time and there are different relationships between new and old symbols.

## 2.3. Symbol-equivariance: SE-RRM

We suggest utilizing symbol equivariance in machine learning models for structured problem-solving tasks and thus extend the vanilla RRM architecture with symbol equivariance, which we denote as SE-RRMs. The core idea of how SE-RRMs are constructed is visualized in Figure 1: SE-RRM introduces a third dimension to link positions and symbols. Analogously to vanilla RRM, SE-RRMs are based on the structurally same fixed-point operation, however with tensors of different shapes. For notational clarity and to better distinguish between the two methods, we denote the SE-RRM fixed-point operation by  $\mathcal{G}$ :

$$Z^{t+1} = \mathcal{G} \left( \mathbf{1}_{\text{emb}} E^{\mathcal{G}}(X) + \mathbf{1}_{\text{prev}} Z^{t-r}, Z^t \right) \quad (8)$$

**Task Embedding.** For SE-RRMs the symbol embedding function  $E^C$  is changed to realize a mapping  $E^C : C^I \rightarrow \mathbb{R}^{D \times I \times K}$ , with  $D$  being the number of features,  $I$  being the number of positions, and  $K \equiv |C|$  being the total number of symbols. Note, that in contrast to vanilla RRM, in principle, we use the same embedding  $\mathbf{d} \in \mathbb{R}^D$  for all of the different symbols across all  $I$  positions. An exception are special symbols like a mask or unknown token, for which we use an

own embedding  $(s_1, \dots, s_n)$ . If a symbol is not present at a certain position, we use a zero embedding ( $\mathbf{0}$ ). Formally this can be considered a function  $h$  realizing:

$$h : C^I \times C \times [I] \rightarrow \mathbb{R}^D$$

$$h(\mathbf{X}, c, i) = \begin{cases} \mathbf{d} \in \mathbb{R}^D, & \text{if } x_i = c \text{ \& } c \in C_{\text{usual}} \\ \mathbf{s}_1 \in \mathbb{R}^D, & \text{if } x_i = c \text{ \& } c = s_1 \\ \vdots \\ \mathbf{s}_n \in \mathbb{R}^D, & \text{if } x_i = c \text{ \& } c = s_n \\ \mathbf{0} \in \mathbb{R}^D, & \text{if } x_i \neq c \end{cases}$$

with  $\mathbf{X} = (x_1, \dots, x_I)$ ,  $C = C_{\text{usual}} \cup \{s_1, \dots, s_n\}$ , where  $s_1, \dots, s_n$  denote special symbols and  $C_{\text{usual}}$  contains all remaining (usual) symbols.  $h$  is uniformly applied across all  $I$  positions and across all  $K$  symbols. Thereby  $\mathbf{d}, \mathbf{s}_1, \dots, \mathbf{s}_n$  are constant vectors.

The extension of  $E^{\mathcal{H}}$  to  $E^{\mathcal{G}}$  for encoding task-type information is different for SE-RRMs compared to vanilla RRM. While we make use of an additional position to encode task types in vanilla RRM, we learn a task-type embedding  $\mathbb{N} \rightarrow \mathbb{R}^{D \times 1 \times K}$  for SE-RRMs. This is broadcasted along the position dimension, giving an effective task-type embedding  $E^{\mathcal{T}} : C^I \times \mathbb{N} \rightarrow \mathbb{R}^{D \times I \times K}$ , which is added to the other embeddings, i.e.,  $E^{\mathcal{G}} := E^C + E^P + E^{\mathcal{T}}$ . We initialize the task-type embeddings to the same vector for all  $K$  symbols. The identical initialization ensures symbol equivariance at the beginning of learning, but allows to deviate for specific task types, where symbol equivariance is not appropriate at the level of single tasks of these specific task types (symbol equivariance would only hold for the whole task type in these cases).

**Adaptation of the output mapping.** In SE-RRM we generate predictions by linearly mapping the feature dimension to a single logit, thus reducing it to a shape of  $(1, I, K)$  or  $(I, K)$ , from which we can either extract the prediction symbol for a position or compute a categorical loss.

**Neural Architecture of SE-RRM blocks.** The initial layer  $H_0$  is analogous to RRM blocks with SE-RRM embeddings and SE-RRM recurrent states being used. In contrast to a single transformer-style layer per block in vanilla RRM, SE-RRM comprises two Transformer-style self-attention layers, where one is applied along the position dimension, while the other is applied along the symbol dimension (Rao et al., 2021; Tolstikhin et al., 2021; Ho et al., 2019; Huang et al., 2019). The update scheme  $\mathcal{G}$  is then as follows:

$$H'_l = \text{Norm}(H_l + T^{D,I}(H_l)), \quad \underline{D} \times \underline{I} \times K \quad (9)$$

$$H''_l = \text{Norm}(H'_l + T^{D,K}(H'_l)), \quad \underline{D} \times I \times \underline{K} \quad (10)$$

$$H_{l+1} = \text{Norm}(H''_l + m^D(H''_l)), \quad \underline{D} \times I \times K \quad (11)$$

for  $L$  additional steps, producing the block output  $\mathbf{Z}^{t+1} = H_L$ . The expressions written to the right of each equation

(e.g.,  $\underline{D} \times \underline{I} \times K$ ) indicate the dimensions of the representations, and along which dimension self-attention is applied.  $T^{D,I}$  indicates that the self-attention layer is applied along the position dimension with the token dimension being the feature dimension.  $T^{D,K}$  is applied along the symbol dimension with the token dimension being the feature dimension. As for vanilla RRM, RMSNorm (Zhang & Sennrich, 2019) is used as a normalization layer, which is applied uniformly across all symbols and positions.

As for vanilla RRM, when omitting positional and task-type encodings in the task embeddings, position equivariance can be shown.

**Proposition 2.2.** *An SE-RRM block  $\mathcal{G}$  without positional embeddings, as defined in Equations (5) and (9) to (11) is equivariant under permutations  $\pi : [I] \rightarrow [I]$  of the input positions  $\in [I] := \{1, \dots, I\}$ , i.e.,*

$$\Pi_2^\pi \left( \mathcal{G} \left( \mathbf{1}_{\text{emb}} E^{\mathcal{G}}(\mathbf{X}) + \mathbf{1}_{\text{prev}} \mathbf{Z}^{t-r}, \mathbf{Z}^t \right) \right) = \mathcal{G} \left( \mathbf{1}_{\text{emb}} E^{\mathcal{G}}(\Pi_2^\pi(\mathbf{X})) + \mathbf{1}_{\text{prev}} \Pi_2^\pi(\mathbf{Z}^{t-r}), \Pi_2^\pi(\mathbf{Z}^t) \right),$$

where  $\Pi_a^\pi$  is defined as for Proposition 2.1.

The proof is analogous to Proposition 2.1, with the only differences being the addition of an extra dimension and the application of a self-attention operation along the symbol dimension in Equation (10). Equation (10) is uniformly applied to all positions and therefore does not break position equivariance. Since all other operations are further uniform across the additional symbol dimension, position equivariance is preserved.

Furthermore, the SE-RRM block exhibits the beneficial property of equivariance under the permutations of symbols.

**Proposition 2.3.** *An SE-RRM block  $\mathcal{G}$  without task-type embeddings, as defined in Equations (5) and (9) to (11) is equivariant under permutations  $\rho : [K] \rightarrow [K]$  of the input symbols  $\in [K] := \{1, \dots, K\}$ , i.e.,*

$$\Pi_3^\rho \left( \mathcal{G} \left( \mathbf{1}_{\text{emb}} E^{\mathcal{G}}(\mathbf{X}) + \mathbf{1}_{\text{prev}} \mathbf{Z}^{t-r}, \mathbf{Z}^t \right) \right) = \mathcal{G} \left( \mathbf{1}_{\text{emb}} E^{\mathcal{G}}(\Pi_3^\rho(\mathbf{X})) + \mathbf{1}_{\text{prev}} \Pi_3^\rho(\mathbf{Z}^{t-r}), \Pi_3^\rho(\mathbf{Z}^t) \right),$$

where  $\Pi_a^\pi$  is defined as for Proposition 2.1.

The proof is analogous to Proposition 2.2 with adaptations according to the application to another dimension. The inclusion of a symbol dimension with an appropriate embedding allows this equivariance.

**Computational Complexity.** An efficient implementation of Attention (Dao et al., 2022) has a computational complexity of  $O(I^2)$  and a memory complexity of  $O(I)$ . For the attention mechanism in SE-RRM this changes to  $O(I^2 K + K^2 I)$  and  $O(IK)$  respectively. For the problems

Table 1. Sudoku performance of different methods across grid sizes. We report *fully solved rate* (FSR), i.e., the percentage of puzzles solved exactly, and *grid-point accuracy* GPA, i.e., the percentage of correctly predicted unfilled Sudoku cells, for each mode with 95% Wilson score confidence intervals. All models except GPT-OSS (OpenAI et al., 2025) are trained on  $9 \times 9$  Sudoku grids. The best RRM is marked bold. SE-RRM generalizes to larger problem sizes, while the other RRM are unable to accommodate unseen symbols.

| Model                    | 4×4 Mini Sudoku                       |                                       | 9×9 Sudoku                            |                                       | 16×16 Maxi Sudoku     |                                       | 25×25 Ultra Sudoku    |                                       |
|--------------------------|---------------------------------------|---------------------------------------|---------------------------------------|---------------------------------------|-----------------------|---------------------------------------|-----------------------|---------------------------------------|
|                          | FSR                                   | GPA                                   | FSR                                   | GPA                                   | FSR                   | GPA                                   | FSR                   | GPA                                   |
| HRM                      | 0 <sup>(0–1.32)</sup>                 | 29.19 <sup>(27.45–30.95)</sup>        | 63.53 <sup>(63.38–63.67)</sup>        | 86.11 <sup>(86.10–86.12)</sup>        | — <sup>a</sup>        | — <sup>a</sup>                        | — <sup>a</sup>        | — <sup>a</sup>                        |
| TRM                      | 0 <sup>(0–1.32)</sup>                 | 45.88 <sup>(43.96–47.79)</sup>        | 71.94 <sup>(71.80–72.08)</sup>        | 89.80 <sup>(89.79–89.81)</sup>        | — <sup>a</sup>        | — <sup>a</sup>                        | — <sup>a</sup>        | — <sup>a</sup>                        |
| <b>SE-RRM (ours)</b>     | <b>95.46</b> <sup>(92.01–97.08)</sup> | <b>99.15</b> <sup>(98.67–99.41)</sup> | <b>93.73</b> <sup>(93.66–93.81)</sup> | <b>97.58</b> <sup>(97.54–97.63)</sup> | 0 <sup>(0–1.75)</sup> | <b>51.95</b> <sup>(51.43–52.47)</sup> | 0 <sup>(0–8.38)</sup> | <b>31.49</b> <sup>(30.22–32.75)</sup> |
| GPT-OSS-20B <sup>b</sup> | 100 <sup>(98.68–100)</sup>            | 100 <sup>(99.85–100)</sup>            | 21.67 <sup>(13.12–33.62)</sup>        | — <sup>c</sup>                        | — <sup>d</sup>        | — <sup>d</sup>                        | — <sup>d</sup>        | — <sup>d</sup>                        |

<sup>a</sup> Cannot extrapolate to this size. <sup>b</sup> LLM-based systems are evaluated without task-specific program synthesis or symbolic search.

<sup>c</sup> The model sometimes refuses to produce any output, thus GPA is therefore not computed. <sup>d</sup> Model refuses to provide solutions.

considered  $I \gg K$ , therefore SE-RRM increases computational and memory complexity linearly by the factor  $K$ . Conversely, in domains where  $K \gg I$ , the computational complexity is dominated by  $O(K^2I)$ , which may render SE-RRM impractical.

### 3. Experiments and Results

We evaluate Hierarchical Reasoning Models (HRMs), Tiny Recursive Models (TRMs), and our proposed Symbol-Equivariant Recurrent Reasoning Models (SE-RRMs) on three classes of structured problem-solving tasks: Sudoku, ARC-AGI, and Maze-solving. All recurrent reasoning models share the same overall training protocol established in HRM (Wang et al., 2025): models are trained end-to-end using supervised learning with deep supervision at each recurrent step, and inference is performed by repeatedly applying the same recurrent block for a fixed number of iterations. Large language model baselines are evaluated without task-specific program synthesis, symbolic search, or human-designed orchestration, reflecting their performance as plain neural solvers. HRM and TRM employ a Q-learning approach that serves as a stopping policy for deep supervision, allowing the model to apply fewer than the maximum number of deep supervision steps during training and thereby reducing computation time. In contrast, SE-RRM uses a simpler stochastic mechanism: at each deep supervision step, training-time supervision is terminated with probability  $p$  (set as a hyperparameter), unless the maximum number of steps has already been reached. During inference, all models — HRM, TRM, and SE-RRM — execute the maximum number of deep supervision steps, which is set to 16 for all experiments in this section.

In addition, we apply RoPE2d (Heo et al., 2024) instead of RoPE to encode the spatial positions  $I$ . A detailed comparison of all hyperparameters together with various ablations can be found in the appendix. Throughout the experiments we use an SE-RRM with only 2 million parameters, which is significantly smaller than the compared methods.

#### 3.1. Sudoku

**Data.** We use a subset of 1,000 samples of the  $9 \times 9$  Sudoku dataset provided by Wang et al. (2025) (HRM), compute 1,000 augmentations per training sample like HRM and TRM, and use the full test set of 422,786 samples for validation. In addition, we test zero-shot generalization to smaller and larger Sudoku puzzles. To this end, we consider  $4 \times 4$ ,  $16 \times 16$ , and  $25 \times 25$  puzzles. For  $4 \times 4$  puzzles, the test set comprises all 288 unique combinations<sup>1</sup>, while we created 216 puzzles with unique solutions of varying difficulty with `py-sudoku` for  $16 \times 16$  Sudoku. Finally, we extracted 42 samples of size  $25 \times 25$  from the curated puzzles in `Sudoku-SMT-Solvers`<sup>2</sup>.

**Approach.** We evaluate all models on Sudoku puzzles of varying grid sizes that range from small  $4 \times 4$  instances to large  $25 \times 25$  puzzles. All RRM (HRM, TRM, and SE-RRM) are trained exclusively on standard  $9 \times 9$  Sudokus and tested without fine-tuning on other grid sizes to assess generalization and extrapolation. Performance is measured using two metrics: *fully solved rate* (FSR), defined as the fraction of puzzles solved exactly, and *grid-point accuracy* (GPA), defined as the fraction of correctly predicted, *initially unfilled* cells. We report 95% confidence intervals using the Wilson score interval.

We use the hyperparameters of the respective papers, thus performing 16 deep supervision steps on inference for HRM, TRM and also SE-RRM. An ablation of the impact of the number of steps on the performance can be found in the appendix.

**Results.** The quantitative evaluation is summarized in Table 1. Here, in the training regime ( $9 \times 9$ ), both HRM and TRM achieve strong performance and outperform GPT-OSS-20B (OpenAI et al., 2025). GPT-OSS-20B — having

<sup>1</sup><https://github.com/Black-Phoenix/4x4-Sudoku-Dataset>

<sup>2</sup><https://github.com/liamjdavis/Sudoku-SMT-Solvers>. Note that the solver `Z3Solver` solved only 42 out of 100 puzzles.

no access to external tools – fails to reliably solve even standard  $9 \times 9$  puzzles despite having been exposed to Sudoku-like problems during pretraining. Our proposed SE-RRM model outperforms all base-lines by a large margin ( $> 11\%$  for FSR and  $> 7\%$  for GPA) proving that reasoning using symbol equivariance is more effective. Note that TRM (Jolicoeur-Martineau, 2025) reports an accuracy of 87.4 % on Sudoku with an MLP-mixer (Tolstikhin et al., 2021), however this version supports only a fixed number of positions and is thus unable to extrapolate to different grids.

Next, we evaluate how the models generalize to simpler  $4 \times 4$  puzzles. HRM and TRM do not extrapolate to  $4 \times 4$  mini Sudoku as the FSR rate drops to zero, showing that both models did not learn the intrinsic Sudoku rules. In contrast, GPT-OSS-20B can perfectly solve these simple logic puzzles. Remarkably, SE-RRM is able to extrapolate the Sudoku rules to  $4 \times 4$  grids and achieves 95.46% FSR and gets almost all unfilled cells right (GPA 99.15%).

Finally, we test generalization to larger grids ( $16 \times 16$  and  $25 \times 25$ ), with  $I$  (number of positions) growing quadratically and  $K$  (number of symbols) growing linearly. HRM and TRM cannot be applied to larger Sudoku grids without any training, as additional symbols ("10" up to "25") would require novel embeddings. Despite numerous attempts, GPT-OSS-20B did not provide any meaningful outputs. This result, along with the performance on  $9 \times 9$  grids, corroborates prior observations that LLMs struggle with tightly constrained symbolic reasoning tasks. Although the trained SE-RRM model does not solve any of the larger Sudokus, the GPA of 51.95% for  $16 \times 16$  grids and 31.49% for  $25 \times 25$  grids is substantially higher than random choices and demonstrates that our trained model is able to generalize beyond the training data distribution.

**Test time scaling.** RRM offers the possibility to adjust the number of steps during inference. For a NP-hard problem like Sudoku, this can improve the performance of the model substantially. Table 2 shows that SE-RRM does not only outperform the other RRM on a different number of deep supervision steps, but also delivers overall the best performance.

Table 2. Test-time scaling on  $9 \times 9$  Sudoku. Fully solved rate (FSR in %) as a function of the number of deep supervision steps at inference.

| Model         | Number of inference steps |       |       |       |       |       |       |       |
|---------------|---------------------------|-------|-------|-------|-------|-------|-------|-------|
|               | 1                         | 2     | 4     | 8     | 16    | 32    | 64    | 128   |
| HRM           | 2.01                      | 20.47 | 49.37 | 59.45 | 63.53 | 65.67 | 67.09 | 68.15 |
| TRM           | 2.51                      | 31.02 | 55.49 | 66.46 | 71.95 | 75.36 | 77.86 | 79.78 |
| SE-RRM (ours) | 16.05                     | 62.06 | 77.31 | 87.38 | 93.73 | 96.82 | 98.22 | 98.84 |

Table 3. pass@2 on ARC-AGI benchmarks and fully solved rate (FSR) on Maze with 95% Wilson score confidence intervals. Reported values for HRM and TRM are taken from the respective papers.

| Model         | ARC-AGI-1                     | ARC-AGI-2                   | Maze                          |
|---------------|-------------------------------|-----------------------------|-------------------------------|
|               | pass@2                        |                             | FSR                           |
| HRM           | 40.3 <sup>(35.56–45.13)</sup> | 5.0 <sup>(2.31–10.48)</sup> | 74.5 <sup>(71.66–77.15)</sup> |
| TRM           | 44.6 <sup>(39.71–49.40)</sup> | 7.8 <sup>(4.0–13.64)</sup>  | 85.3 <sup>(82.92–87.41)</sup> |
| SE-RRM (ours) | 45.3 <sup>(40.44–50.15)</sup> | 7.1 <sup>(3.42–12.61)</sup> | 88.8 <sup>(86.64–90.65)</sup> |

### 3.2. ARC-AGI

**Data.** Throughout this section, we use the official training and evaluation sets of ARC-AGI-1 (Chollet, 2019) and ARC-AGI-2 (Chollet et al., 2025), consisting of geometric puzzles. Each puzzle uses at most 10 colors distributed across a grid of at most  $30 \times 30$  positions and 2-11 few-shot examples demonstrate the underlying task. Like HRM and TRM, we include 160 puzzles of the ConceptARC-dataset (Moskvichev et al., 2023) and all few-shot examples of the evaluation puzzles for training. Therefore, the training set comprises 960 and 1,280 puzzles for ARC-AGI-1 and ARC-AGI-2, respectively. The resulting models are evaluated on the official 400 test puzzles for ARC-AGI-1 and 120 test puzzles of the ARC-AGI-2 challenge.

**Approach.** We compare the officially reported performance values of HRM, TRM, and SE-RRM. In contrast to the extensive color augmentations applied by HRM and TRM, we only use 8 dihedral augmentations for each puzzle due to symbol equivariance of SE-RRM. Moreover, several ablations, where we incrementally apply the hyperparameters of SE-RRM to TRM, are presented in the appendix.

**Results.** SE-RRM outperforms HRM on both ARC-AGI-1 and ARC-AGI-2 and delivers comparable results to TRM. In Table 3, we report results from the original publications of HRM and TRM, compared with SE-RRM.

### 3.3. Maze

**Data.** Finally, we evaluate the models on grid-based maze-solving tasks, which require multi-step planning, global constraint satisfaction, and consistent state tracking. All models are trained on the Maze-hard dataset defined by Wang et al. (2025), consisting of 1,000 training and 1,000 test mazes. Each maze is defined on a  $30 \times 30$  grid and has a minimal solution path length of 110. Note that we do not perform any augmentations during training of SE-RRM.

**Approach.** In contrast to the previous puzzles, symbol equivariance is not reasonable for this task, as walls are not equivalent to start and end points. Thus, we allow different embeddings for each symbol, breaking the symbol equivariance. The resulting model is neither equivariant towards

permutations of symbols nor to permutations of positions.

**Results.** Consistent with the results on Sudoku and ARC-AGI, SE-RRM achieves competitive results on Maze (Table 3). SE-RRM with broken symbol-equivariance performs slightly better than TRM and much better than HRM. This could be due to the higher-dimensional input embedding or the additional transformer layer in an SE-RRM block. In any case, it shows that SE-RRM can be successfully applied to problems that do not require symbol equivariance.

## 4. Conclusion and Limitations

Overall, across all three domains, we show that symbol-equivariant recurrent reasoning models offer a robust and scalable alternative to both symbolic solvers and LLM-based approaches for structured problem-solving tasks. We show that SE-RRMs require substantially fewer augmentations. Specifically, we trained on ARC-AGI with 8 instead of 1000 augmentations per sample. On Sudoku, where no compared model applies extensive permutations of symbols as data augmentation, SE-RRM delivers superior results. Moreover, our architecture enables recurrent reasoning models to accommodate novel symbols, which allows extrapolation to different task formulations.

**Limitations.** Throughout the paper, we only used SE-RRMs with 2M parameters. This is substantially fewer than TRM (7M) or HRM (27M). The reduced parameter count can partially offset the increased computational and memory complexity during training and inference.

## Acknowledgements

We thank Sebastian Lehner for helpful discussions. The ELLIS Unit Linz, the LIT AI Lab, the Institute for Machine Learning, are supported by the Federal State Upper Austria. We thank the projects FWF Bilateral Artificial Intelligence (10.55776/COE12), FWF AIRI FG 9-N (10.55776/FG9), AI4GreenHeatingGrids (FFG- 899943), Stars4Waters (HORIZON-CL6-2021- CLIMATE-01-01). We also thank NXAI GmbH, Audi AG, Merck Healthcare KGaA, TÜV Holding GmbH, Software Competence Center Hagenberg GmbH, dSPACE GmbH, TRUMPF SE + Co. KG.

## Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

## References

- Almeida, L. B. A learning rule for asynchronous perceptrons with feed-back in a combinatorial environment. In *IEEE First International Conference on Neural Networks*, volume 2, pp. 609–618. IEEE, 1987.
- Anil, C., Wu, Y., Andreassen, A., Lewkowycz, A., Misra, V., Ramasesh, V., Slone, A., Gur-Ari, G., Dyer, E., and Neyshabur, B. Exploring length generalization in large language models. *Advances in Neural Information Processing Systems*, 35:38546–38556, 2022.
- ARC Prize Team. The hidden drivers of hrm’s performance on arc-agi. <https://arcprize.org/blog/hrm-analysis>, 2025. Accessed: 2025-01-27.
- Bai, S., Kolter, J. Z., and Koltun, V. Deep equilibrium models. *Advances in Neural Information Processing Systems*, 32, 2019.
- Bartmann, C., Schimunek, J., Ielanskyi, M., Seidl, P., Klambauer, G., and Luukkonen, S. Moleculariq: Characterizing chemical reasoning capabilities through symbolic verification on molecular graphs. *International Conference On Learning Representations (ICLR)*, 2026.
- Bengio, Y., Lodi, A., and Prouvost, A. Machine learning for combinatorial optimization: a methodological tour d’horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.
- Biere, A., Heule, M., van Maaren, H., and Walsh, T. (eds.). *Handbook of Satisfiability - Second Edition*, volume 336 of *Frontiers in Artificial Intelligence and Applications*. IOS Press, 2021. ISBN 978-1-64368-160-3. doi: 10.3233/FAIA336. URL <https://doi.org/10.3233/FAIA336>.
- Bronstein, M. M., Bruna, J., LeCun, Y., Szlam, A., and Vandergheynst, P. Geometric deep learning: going beyond euclidean data. *IEEE Signal Processing Magazine*, 34(4): 18–42, 2017.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33: 1877–1901, 2020.
- Chollet, F. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.
- Chollet, F., Knoop, M., Kamradt, G., Landers, B., and Pinkard, H. Arc-agi-2: A new challenge for frontier ai reasoning systems. *arXiv preprint arXiv:2505.11831*, 2025.

- Cohen, T. and Welling, M. Group equivariant convolutional networks. In *International Conference on Machine Learning*, pp. 2990–2999. PMLR, 2016.
- Cole, J. and Osman, M. Don’t throw the baby out with the bathwater: How and why deep learning for arc. *arXiv preprint arXiv:2506.14276*, 2025.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022.
- Defferrard, M., Bresson, X., and Vandergheynst, P. Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering. In Lee, D., Sugiyama, M., Luxburg, U., Guyon, I., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 29, 2016.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186, 2019.
- Ellis, K., Wong, C., Nye, M., Sable-Meyer, M., Cary, L., Morales, L., Hewitt, L., Solar-Lezama, A., and Tenenbaum, J. B. Dreamcoder: Growing generalizable, interpretable knowledge with wake-sleep bayesian program learning. *arXiv preprint arXiv:2006.08381*, 2020.
- Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., and Dahl, G. E. Neural Message Passing for Quantum Chemistry. In Precup, D. and Teh, Y. W. (eds.), *International Conference on Machine Learning*, volume 70, pp. 1263–1272, 2017.
- Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X., et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025.
- Hazra, R., Venturato, G., Martires, P. Z. D., and De Raedt, L. Can large language models reason? a characterization via 3-sat. *arXiv preprint arXiv:2408.07215*, 2024.
- Heo, B., Park, S., Han, D., and Yun, S. Rotary position embedding for vision transformer. In *European Conference on Computer Vision*, pp. 289–305. Springer, 2024.
- Ho, J., Kalchbrenner, N., Weissenborn, D., and Salimans, T. Axial attention in multidimensional transformers. *arXiv preprint arXiv:1912.12180*, 2019.
- Huang, Z., Wang, X., Huang, L., Huang, C., Wei, Y., and Liu, W. Ccnet: Criss-cross attention for semantic segmentation. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 603–612, 2019.
- Ilse, M., Tomczak, J., and Welling, M. Attention-based deep multiple instance learning. In *International Conference on Machine Learning*, pp. 2127–2136. PMLR, 2018.
- Jolicoeur-Martineau, A. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*, 2025.
- Jordan, M. Serial order: A parallel distributed processing approach. *ICS Report*, 1986.
- Kazemnejad, A., Padhi, I., Natesan Ramamurthy, K., Das, P., and Reddy, S. The impact of positional encoding on length generalization in transformers. *Advances in Neural Information Processing Systems*, 36:24892–24928, 2023.
- Kim, J., Podlasek, A., Shidara, K., Liu, F., Alaa, A., and Bernardo, D. Limitations of large language models in clinical problem-solving arising from inflexible reasoning. *Scientific reports*, 15(1):39426, 2025.
- Kipf, T. N. and Welling, M. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*, 2017.
- Kirsch, L., Harrison, J., Freeman, C. D., Sohl-Dickstein, J., and Schmidhuber, J. Towards general-purpose in-context learning agents. In *NeurIPS 2023 Foundation Models for Decision Making Workshop*, 2023. URL <https://openreview.net/forum?id=zDTqQVGgzH>.
- Knuth, D. E. *The Art of Computer Programming, Volume 4, Fascicle 6. Satisfiability*. Addison-Wesley Professional, 2015.
- Kotary, J., Fioretto, F., Van Hentenryck, P., and Wilder, B. End-to-end constrained optimization learning: A survey. *arXiv preprint arXiv:2103.16378*, 2021.
- Lee, J., Lee, Y., Kim, J., Kosiorek, A., Choi, S., and Teh, Y. W. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International Conference on Machine Learning*, pp. 3744–3753. PMLR, 2019.
- Li, W.-D., Hu, K., Larsen, C., Wu, Y., Alford, S., Woo, C., Dunn, S. M., Tang, H., Naim, M., Nguyen, D., et al. Combining induction and transduction for abstract reasoning. *arXiv preprint arXiv:2411.02272*, 2024.
- Liao, Q. and Poggio, T. Simple recursive model: Simplified, single-state reasoning with skip connections. Research Square preprint, 2026. URL <https://www.researchsquare.com/article/rs-8492126/v1>. Version 1 posted January 7, 2026.
- Long, J. Large language model guided tree-of-thought. *arXiv preprint arXiv:2305.08291*, 2023.

- Malek, A., Ge, J., Lazic, N., Jin, C., György, A., and Szepesvári, C. Frontier llms still struggle with simple reasoning tasks. *arXiv preprint arXiv:2507.07313*, 2025.
- Merrill, W., Petty, J., and Sabharwal, A. The illusion of state in state-space models. *arXiv preprint arXiv:2404.08819*, 2025.
- Mirchandani, S., Xia, F., Florence, P., Ichter, B., Driess, D., Arenas, M. G., Rao, K., Sadigh, D., and Zeng, A. Large language models as general pattern machines. *arXiv preprint arXiv:2307.04721*, 2023.
- Mirza, A., Alampara, N., Kunchapu, S., Ríos-García, M., Emoekabu, B., Krishnan, A., Gupta, T., Schilling-Wilhelmi, M., Okereke, M., Aneesh, A., et al. A framework for evaluating the chemical knowledge and reasoning abilities of large language models against the expertise of chemists. *Nature Chemistry*, pp. 1–8, 2025.
- Moskvichev, A., Odouard, V. V., and Mitchell, M. The conceptarc benchmark: Evaluating understanding and generalization in the arc domain. *arXiv preprint arXiv:2305.07141*, 2023.
- OpenAI, Agarwal, S., Ahmad, L., Ai, J., Altman, S., Applebaum, A., Arbus, E., Arora, R. K., Bai, Y., Baker, B., Bao, H., Barak, B., Bennett, A., Bertao, T., Brett, N., Brevdo, E., Brockman, G., Bubeck, S., Chang, C., Chen, K., Chen, M., Cheung, E., Clark, A., Cook, D., Dukhan, M., Dvorkak, C., Fives, K., Fomenko, V., Garipov, T., Georgiev, K., Glaese, M., Gogineni, T., Goucher, A., Gross, L., Guzman, K. G., Hallman, J., Hehir, J., Heidecke, J., Helyar, A., Hu, H., Huet, R., Huh, J., Jain, S., Johnson, Z., Koch, C., Kofman, I., Kundel, D., Kwon, J., Kyrilov, V., Le, E. Y., Leclerc, G., Lennon, J. P., Lessans, S., Lezcano-Casado, M., Li, Y., Li, Z., Lin, J., Liss, J., Lily, Liu, Liu, J., Lu, K., Lu, C., Martinovic, Z., McCallum, L., McGrath, J., McKinney, S., McLaughlin, A., Mei, S., Mostovoy, S., Mu, T., Myles, G., Neitz, A., Nichol, A., Pachocki, J., Paino, A., Palmie, D., Pantuliano, A., Parascandolo, G., Park, J., Pathak, L., Paz, C., Peran, L., Pimenov, D., Pokrass, M., Proehl, E., Qiu, H., Raila, G., Raso, F., Ren, H., Richardson, K., Robinson, D., Rotsted, B., Salman, H., Sanjeev, S., Schwarzer, M., Sculley, D., Sikchi, H., Simon, K., Singhal, K., Song, Y., Stuckey, D., Sun, Z., Tillet, P., Toizer, S., Tsimplouras, F., Vyas, N., Wallace, E., Wang, X., Wang, M., Watkins, O., Weil, K., Wendling, A., Whinnery, K., Whitney, C., Wong, H., Yang, L., Yang, Y., Yasunaga, M., Ying, K., Zaremba, W., Zhan, W., Zhang, C., Zhang, B., Zhang, E., and Zhao, S. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.
- Padayachy, K., Richman, R., and Wüthrich, M. V. Tab-trm: Tiny recursive model for insurance pricing on tabular data. *arXiv preprint arXiv:2601.07675*, 2026.
- Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K., and Zettlemoyer, L. Deep contextualized word representations. *arXiv preprint arXiv:1802.05365*, 2018.
- Pineda, F. Generalization of back propagation to recurrent and higher order neural networks. In *Neural information processing systems*, 1987.
- Poetiq. Poetiq shatters ARC-AGI-2 state of the art at half the cost. Poetiq AI blog post, December 2025. URL [https://poetiq.ai/posts/arcagi\\_verified/](https://poetiq.ai/posts/arcagi_verified/). Accessed: 2026-01-13.
- Pourcel, J., Colas, C., and Oudeyer, P.-Y. Self-improving language models for evolutionary program synthesis: A case study on arc-agi. *arXiv preprint arXiv:2507.14172*, 2025.
- Rao, R. M., Liu, J., Verkuil, R., Meier, J., Canny, J., Abbeel, P., Sercu, T., and Rives, A. Msa transformer. In *International Conference on Machine Learning*, pp. 8844–8856. PMLR, 2021.
- Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., and Monfardini, G. The Graph Neural Network Model. *IEEE transactions on neural networks*, 20(1):61–80, 2008.
- Schmidhuber, J. Deep learning in neural networks: An overview. *Neural networks*, 61:85–117, 2015.
- Seely, J., Imajuku, Y., Zhao, T., Cetin, E., and Jones, L. Sudoku-bench: Evaluating creative reasoning with sudoku variants. *arXiv preprint arXiv:2505.16135*, 2025.
- Shazeer, N. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- Sinii, V., Nikulin, A., Kurenkov, V., Zisman, I., and Kolesnikov, S. In-context reinforcement learning for variable action spaces, 2024. URL <https://arxiv.org/abs/2312.13327>.
- Srivastava, A., Rastogi, A., Rao, A., Shoeb, A. A. M., Abid, A., Fisch, A., Brown, A. R., Santoro, A., Gupta, A., Garriga-Alonso, A., et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on machine learning research*, 2023.
- Su, J., Lu, Y., Pan, S., Wen, B., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.
- Tolstikhin, I. O., Houlsby, N., Kolesnikov, A., Beyer, L., Zhai, X., Unterthiner, T., Yung, J., Steiner, A., Keysers, D., Uszkoreit, J., et al. Mlp-mixer: An all-mlp architecture for vision. *Advances in Neural Information Processing Systems*, 34:24261–24272, 2021.

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
- Wang, G., Li, J., Sun, Y., Chen, X., Liu, C., Wu, Y., Lu, M., Song, S., and Yadkori, Y. A. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*, 2025.
- Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022a.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35: 24824–24837, 2022b.
- Xu, X., Xu, Q., Xiao, T., Chen, T., Yan, Y., Zhang, J., Diao, S., Yang, C., and Wang, Y. Ugphysics: A comprehensive benchmark for undergraduate physics reasoning with large language models. *arXiv preprint arXiv:2502.00334*, 2025.
- Yato, T. and Seta, T. Complexity and completeness of finding another solution and its application to puzzles. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, 86(5):1052–1060, 2003.
- Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R. R., and Smola, A. Deep sets. In Guyon, I., Luxburg, U. V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL [https://proceedings.neurips.cc/paper\\_files/paper/2017/file/f22e4747da1aa27e363d86d40ff442fe-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2017/file/f22e4747da1aa27e363d86d40ff442fe-Paper.pdf).
- Zhang, B. and Sennrich, R. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32, 2019.
- Zhang, J., Liu, C., Li, X., Zhen, H.-L., Yuan, M., Li, Y., and Yan, J. A survey for solving mixed integer programming via machine learning. *Neurocomputing*, 519:205–217, 2023.

## A. Notation

Table A1. Notation used in the description of Recurrent Reasoning Models (RRMs), Hierarchical Reasoning Models (HRMs), Tiny Recursive Models (TRMs), and Symbol-Equivariant RRM (SE-RRMs).

| Definition                                     | Symbol                              | Type                                                                                                                    |
|------------------------------------------------|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Scalars and indices</b>                     |                                     |                                                                                                                         |
| feature dimension                              | $D$                                 | $\mathbb{N}$                                                                                                            |
| number of positions (grid points)              | $I$                                 | $\mathbb{N}$                                                                                                            |
| number of symbols / colors                     | $K$                                 | $\mathbb{N}$                                                                                                            |
| recurrent iteration index                      | $t$                                 | $\mathbb{N}$                                                                                                            |
| number of layers per block                     | $L$                                 | $\mathbb{N}$                                                                                                            |
| final iteration step                           | $T$                                 | $\mathbb{N}$                                                                                                            |
| number of additional task-type positions (RRM) | $P$                                 | $\mathbb{N}$                                                                                                            |
| <b>Sets and spaces</b>                         |                                     |                                                                                                                         |
| symbol / color alphabet                        | $C$                                 | finite set, $ C  = K$                                                                                                   |
| input and solution space                       | $C^I$                               | tuples of symbols                                                                                                       |
| <b>Tasks and solutions</b>                     |                                     |                                                                                                                         |
| input task                                     | $\mathbf{X} = (x_1, \dots, x_I)$    | $C^I$                                                                                                                   |
| solution task                                  | $\mathbf{Y} = (y_1, \dots, y_I)$    | $C^I$                                                                                                                   |
| symbol at position $i$                         | $x_i, y_i$                          | $C$                                                                                                                     |
| task-type identifier                           | $p$                                 | $\mathbb{N}$                                                                                                            |
| <b>Embeddings and representations</b>          |                                     |                                                                                                                         |
| symbol-level embedding (SE-RRM)                | $E^C$                               | $C^I \rightarrow \mathbb{R}^{D \times I \times K}$                                                                      |
| positional embedding (SE-RRM)                  | $E^P$                               | $C^I \rightarrow \mathbb{R}^{D \times I \times K}$                                                                      |
| task-type embedding (SE-RRM)                   | $E^T$                               | $C^I \times \mathbb{N} \rightarrow \mathbb{R}^{D \times 1 \times K}$                                                    |
| combined task embedding (SE-RRM)               | $E^G$                               | $E^C + E^P + E^T$                                                                                                       |
| combined task embedding (RRM)                  | $E^H$                               | $C^I \rightarrow \mathbb{R}^{D \times I}$ or $C^I \times \mathbb{N} \rightarrow \mathbb{R}^{D \times (I+P)}$            |
| SE-RRM symbol embedding function               | $h$                                 | $C^I \times C \times [I] \rightarrow \mathbb{R}^D$                                                                      |
| shared usual-symbol embedding vector           | $\mathbf{d}$                        | $\mathbb{R}^D$                                                                                                          |
| special-symbol embedding vectors               | $\mathbf{s}_1, \dots, \mathbf{s}_n$ | $\mathbb{R}^D$                                                                                                          |
| HRM/TRM task embedding matrix                  | $\mathbf{E} = E^H(\mathbf{X})$      | $\mathbb{R}^{D \times I}$                                                                                               |
| SE-RRM task embedding tensor                   | $\mathbf{E}' = E^G(\mathbf{X})$     | $\mathbb{R}^{D \times I \times K}$                                                                                      |
| token embedding at position $i$                | $e_i$                               | $\mathbb{R}^D$                                                                                                          |
| intermediate block states                      | $H_l, H'_l, H''_l$                  | $\mathbb{R}^{D \times I}$ or $\mathbb{R}^{D \times I \times K}$                                                         |
| recurrent state at step $t$                    | $\mathbf{Z}^t$                      | $\mathbb{R}^{D \times I}$ or $\mathbb{R}^{D \times I \times K}$                                                         |
| <b>Models and blocks</b>                       |                                     |                                                                                                                         |
| RRM block (HRM/TRM)                            | $\mathcal{H}$                       | $\mathbb{R}^{D \times I} \times \mathbb{R}^{D \times I} \rightarrow \mathbb{R}^{D \times I}$                            |
| SE-RRM block                                   | $\mathcal{G}$                       | $\mathbb{R}^{D \times I \times K} \times \mathbb{R}^{D \times I \times K} \rightarrow \mathbb{R}^{D \times I \times K}$ |
| embedding inclusion indicator                  | $\mathbf{1}_{\text{emb}}$           | $\{0, 1\}$                                                                                                              |
| previous-state inclusion indicator             | $\mathbf{1}_{\text{prev}}$          | $\{0, 1\}$                                                                                                              |
| <b>Neural operators</b>                        |                                     |                                                                                                                         |
| self-attention over positions                  | $T^{D,I}$                           | $\mathbb{R}^{D \times I} \rightarrow \mathbb{R}^{D \times I}$                                                           |
| self-attention over symbols                    | $T^{D,K}$                           | $\mathbb{R}^{D \times K} \rightarrow \mathbb{R}^{D \times K}$                                                           |
| MLP over features                              | $m^D$                               | $\mathbb{R}^D \rightarrow \mathbb{R}^D$                                                                                 |
| normalization (RMSNorm)                        | Norm                                | $\mathbb{R}^D \rightarrow \mathbb{R}^D$                                                                                 |
| <b>Prediction and loss</b>                     |                                     |                                                                                                                         |
| output mapping (RRM)                           | $\mathbf{O}$                        | $\mathbb{R}^{D \times I} \rightarrow \mathbb{R}^{K \times I}$                                                           |
| output projection matrix (RRM)                 | $\mathbf{W}$                        | $\mathbb{R}^{K \times D}$                                                                                               |
| output projection matrix (SE-RRM)              | $\mathbf{W}$                        | $\mathbb{R}^{1 \times D}$                                                                                               |
| logits at step $t$                             | $\hat{\mathbf{Y}}^t$                | $\mathbb{R}^{K \times I}$ (RRM) or $\mathbb{R}^{I \times K}$ (SE-RRM)                                                   |
| categorical cross-entropy loss                 | $L(\hat{\mathbf{Y}}, \mathbf{Y})$   | $\mathbb{R}_{\geq 0}$                                                                                                   |
| <b>Symmetry operators</b>                      |                                     |                                                                                                                         |
| permutation of positions                       | $\pi$                               | bijection on $\{1, \dots, I\}$                                                                                          |
| permutation of symbols/colors                  | $\rho$                              | bijection on $\{1, \dots, K\}$                                                                                          |
| permutation operator on tensor                 | $\Pi_a^\pi$                         | permutes axis $a$ with permutation $\pi$                                                                                |
| <b>Task-type components</b>                    |                                     |                                                                                                                         |
| task-type embedding position (RRM)             | $\mathbf{p}$                        | $\mathbb{R}^D$                                                                                                          |
| task-type embedding matrix (SE-RRM)            | $\mathbf{P}$                        | $\mathbb{R}^{D \times K}$                                                                                               |

## B. Symbol Equivariance

Based on a small  $4 \times 4$  Sudoku we intuitively explain the concept of symbol equivariance in Figure A1. Moreover, we show a simple exemplary ARC-AGI example in Figure A2.

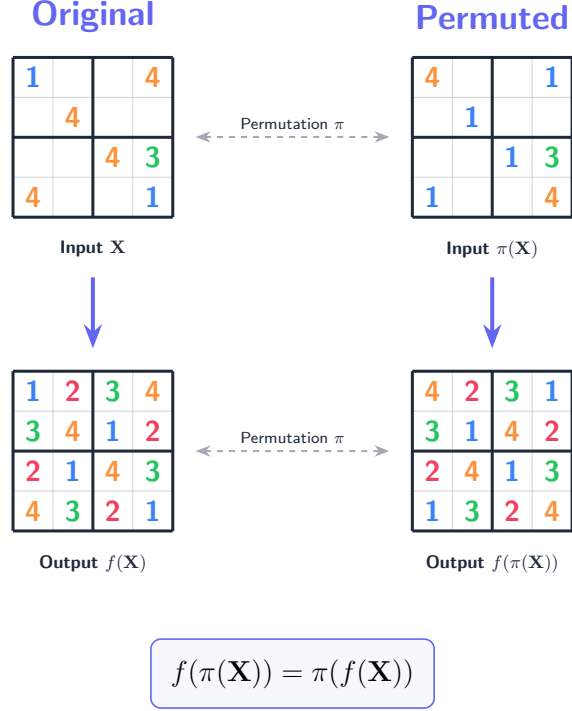


Figure A1. Illustration of Model Equivariance under Symbol Permutation. The model  $f$  is equivariant if applying a permutation  $\pi$  to the input  $X$  results in an equivalent permutation of the output, such that  $f(\pi(X)) = \pi(f(X))$ . Here, the permutation  $\pi$  swaps the symbols 1 and 4. In contrast to other reasoning models, SE-RRM guarantees permutation equivariance.

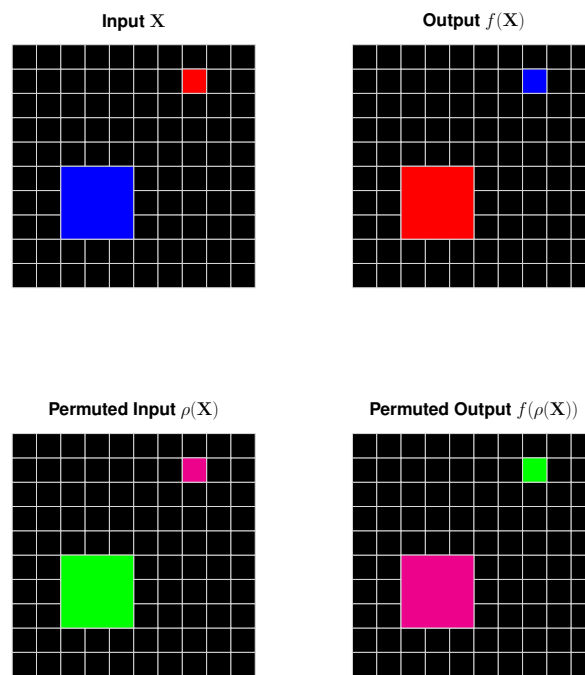


Figure A2. Illustration of an exemplary ARC-AGI task where the colors of the squares in the input have to be exchanged. Vanilla RRM would rely on extensive colour augmentations, while SE-RRMs can immediately learn the task type rule.

## C. Architecture and hyperparameters

### C.1. Sudoku

Table A2. Hyperparameter Comparison for Sudoku

| Hyperparameter                            | HRM                  | TRM                  | SE-RRM                      |
|-------------------------------------------|----------------------|----------------------|-----------------------------|
| <i>Deep supervision steps</i>             | 16                   | 16                   | 16                          |
| <i>early stopping of deep supervision</i> | q-learning           | q-learning           | random (5% chance per step) |
| <i>Recursive Cycles</i>                   | $H = 2, L = 2$       | $H = 3, L = 6$       | $H = 3, L = 6$              |
| <i>Batch Size</i>                         | 384                  | 768                  | 272                         |
| <i>Number of epochs</i>                   | 50000                | 50000                | 10000                       |
| <i>Learning Rate</i>                      | 0.0001               | 0.0001               | 0.0005                      |
| <i>Weight Decay</i>                       | 1                    | 1                    | 1                           |
| <i>LR Schedule</i>                        | Warmup + Constant-LR | Warmup + Constant-LR | Warmup + Constant-LR        |
| <i>Hidden Size</i>                        | 512                  | 512                  | 256                         |
| <i>Parameters</i>                         | 27M                  | 7M                   | 2M                          |
| <i>Transformer Layers</i>                 | 8                    | 2                    | 4 (2 for each dimension)    |
| <i>Positional Encoding</i>                | RoPE                 | RoPE                 | RoPE2D                      |

### C.2. ARC-AGI

Table A3. Hyperparameter Comparison for ARC-AGI. All models use the same hyperparameters for ARC-AGI-1 and ARC-AGI-2.

| Hyperparameter                             | HRM                  | TRM                  | SE-RRM                       |
|--------------------------------------------|----------------------|----------------------|------------------------------|
| <i>Deep supervision steps</i>              | 16                   | 16                   | 16                           |
| <i>early stopping of deep supervision</i>  | q-learning           | q-learning           | random (10% chance per step) |
| <i>Recursive Cycles</i>                    | $H = 2, L = 2$       | $H = 3, L = 4$       | $H = 3, L = 4$               |
| <i>Batch Size</i>                          | 768                  | 768                  | 272                          |
| <i>Number of epochs</i>                    | 100000               | 100000               | 100000                       |
| <i>Learning Rate</i>                       | 0.0001               | 0.0001               | 0.0005                       |
| <i>Weight Decay</i>                        | 0.1                  | 0.1                  | 0.1                          |
| <i>LR Schedule</i>                         | Warmup + Constant-LR | Warmup + Constant-LR | Warmup + Cosine-Decay        |
| <i>Learning Rate for puzzle embeddings</i> | 0.01                 | 0.01                 | 0.01                         |
| <i>Weight Decay for puzzle embeddings</i>  | 0.1                  | 0.1                  | 0.3                          |
| <i>Hidden Size</i>                         | 512                  | 512                  | 256                          |
| <i>Parameters</i>                          | 27M                  | 7M                   | 2M                           |
| <i>Transformer Layers</i>                  | 8                    | 2                    | 4 (2 for each dimension)     |
| <i>Positional Encoding</i>                 | RoPE                 | RoPE                 | RoPE2D                       |

### C.3. Maze

Table A4. Hyperparameter Comparison for Maze

| Hyperparameter                     | HRM                  | TRM                  | SE-RRM                       |
|------------------------------------|----------------------|----------------------|------------------------------|
| Deep supervision steps             | 16                   | 16                   | 16                           |
| early stopping of deep supervision | q-learning           | q-learning           | random (10% chance per step) |
| Recursive Cycles                   | $H = 2, L = 2$       | $H = 3, L = 4$       | $H = 3, L = 4$               |
| Batch Size                         | 384                  | 768                  | 64                           |
| Number of epochs                   | 20000                | 50000                | 10000                        |
| Learning Rate                      | 0.0001               | 0.0001               | 0.0005                       |
| Weight Decay                       | 1                    | 1                    | 1                            |
| LR Schedule                        | Warmup + Constant-LR | Warmup + Constant-LR | Warmup + Constant-LR         |
| Hidden Size                        | 512                  | 512                  | 256                          |
| Parameters                         | 27M                  | 7M                   | 2M                           |
| Transformer Layers                 | 8                    | 2                    | 4 (2 for each dimension)     |
| Positional Encoding                | RoPE                 | RoPE                 | RoPE2D                       |

## D. Additional experimental results

### D.1. Ablation on ARC-AGI-1

In Table A5, we gradually transform TRM into SE-RRM and show the implications on the ARC-AGI-1 benchmark. First, we replace Q-learning with the random halting mechanism of deep supervision (TRM + random halting). Then, we change RoPE to RoPE2d. Interestingly, both of these changes have a clear negative effect on TRM’s performance. When we additionally match all remaining hyperparameters, this effect is even more pronounced. However, this is less surprising, as SE-RRM also has significantly fewer parameters. After further adding the 2-dimensional embedding and adopting our architecture, we arrive at SE-RRM.

Table A5. pass@2 on ARC-AGI-1 benchmark with 95% Wilson score confidence intervals. The hyperparameters and architecture of TRM is changed step by step into SE-RRM.

| Model                                                            | ARC-AGI-1 (%)                        |
|------------------------------------------------------------------|--------------------------------------|
| TRM                                                              | 44.6 <sup>(39.71–49.40)</sup>        |
| TRM + random halting                                             | 34.8 <sup>(30.25–39.54)</sup>        |
| TRM + RoPE2d                                                     | 38.4 <sup>(33.62–43.10)</sup>        |
| TRM + random halting + RoPE2d                                    | 33.1 <sup>(28.57–37.75)</sup>        |
| TRM + all hyperparameters matched                                | 32.3 <sup>(27.86–36.98)</sup>        |
| TRM + all hyperparameters matched + equivariance = SE-RRM (ours) | <b>45.3</b> <sup>(40.44–50.15)</sup> |

### D.2. Learning dynamics

In Figure A3 we compare the development of the Pass@2 accuracy on ARC-AGI1 and FSR (Fully Solved Rate) on Sudoku and Maze of HRM, SE-RRM and TRM during training on different datasets.

### D.3. Ablation on Sudoku and Maze

We further evaluate the different choices of hyperparameters on Sudoku and Maze. More specifically, we train TRM both with random halting and q-learning, evaluate both models with RoPE and RoPE2D, and try hyperparameters of TRM on SE-RRM and the other way round. Note that we do not try q-learning with SE-RRM, as the encoding has a different shape which prevents a straight-forward implementation. Results are shown in Table A6. While SE-RRM seems to work quite well under different configurations, TRM sometimes does not work at all. We suspect that the hyperparameters of TRM are optimized for q-learning and don’t transfer well to different settings.

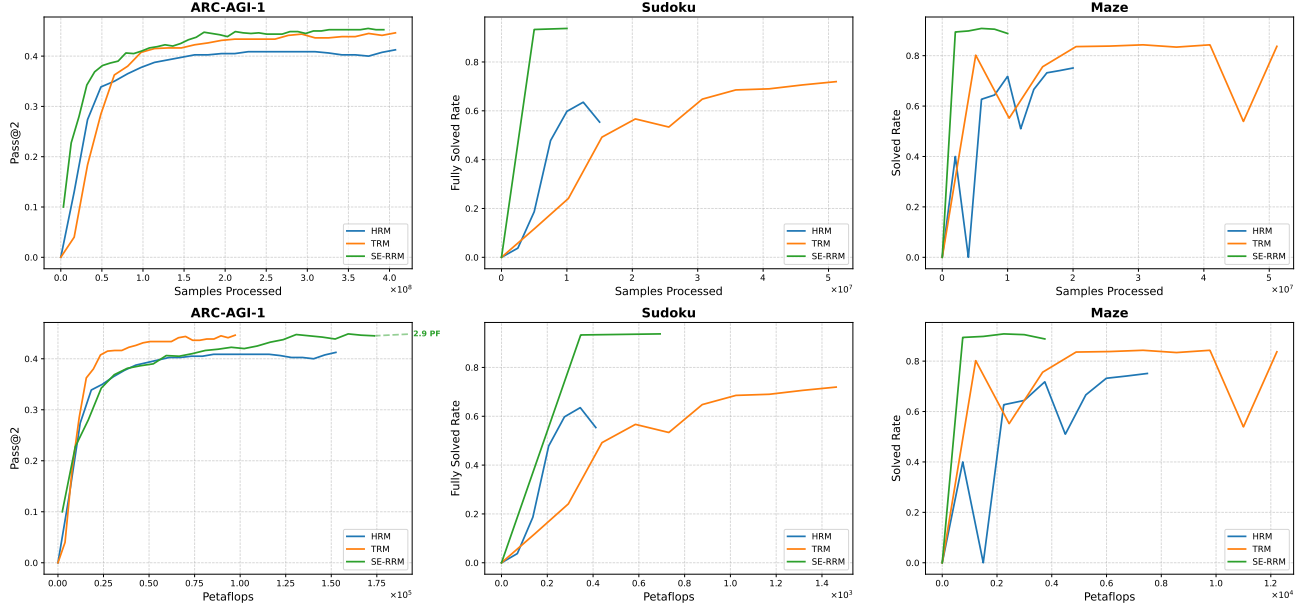


Figure A3. Learning curves for HRM, TRM, and SE-RRM. Validation metrics HRM, TRM and SE-RRM on Sudoku, ARC-AGI-1 and Maze are shown on the y-axis. The number of samples processes during training is shown on the x-axis in the first row and the Petaflops are shown on the x-axis in the second row. SE-RRM is more data efficient since it reaches higher metrics with lower number of samples processed.

In Table A7 we ablate the effect of enforced equivariance via shared embeddings. SE-RRM performs best on Sudoku with enforced equivariance and slightly worse without. Note that equivariance can still be learned during training, even if the initial embeddings for every digit are different. In Maze, the symbols have a clearly different semantics. Therefore, enforcing equivariance leads to inferior performance.

Table A6. Ablations of TRM and SE-RRM on Sudoku and Maze.

| Model  | Halting    | PosEnc | Hyperparameters | Sudoku (%)                           | Maze (%)                             |
|--------|------------|--------|-----------------|--------------------------------------|--------------------------------------|
| TRM    | q-learning | RoPE   | TRM             | 71.9 <sup>(71.80–72.08)</sup>        | 85.3 <sup>(82.92–87.41)</sup>        |
| TRM    | random     | RoPE   | TRM             | 79.4 <sup>(79.31–79.55)</sup>        | 84.9 <sup>(82.55–86.99)</sup>        |
| TRM    | q-learning | RoPE2D | TRM             | 76.5 <sup>(76.34–76.59)</sup>        | 85.4 <sup>(83.08–87.45)</sup>        |
| TRM    | random     | RoPE2D | TRM             | 0 <sup>(0.00–0.00)</sup>             | 59.3 <sup>(56.23–62.30)</sup>        |
| TRM    | random     | RoPE2D | SE-RRM          | 0 <sup>(0.00–0.00)</sup>             | 65.0 <sup>(61.99–67.89)</sup>        |
| SE-RRM | random     | RoPE2D | SE-RRM          | 93.7 <sup>(93.66–93.81)</sup>        | <b>88.8</b> <sup>(86.64–90.65)</sup> |
| SE-RRM | random     | RoPE   | SE-RRM          | <b>95.4</b> <sup>(95.32–95.45)</sup> | 78.3 <sup>(75.64–80.74)</sup>        |
| SE-RRM | random     | RoPE   | TRM             | 68.4 <sup>(68.30–68.58)</sup>        | 80.1 <sup>(77.51–82.46)</sup>        |

*Table A7.* Results of SE-RRM on Sudoku and Maze including ablations. The second column indicates whether symbol equivariance is enforced through shared embeddings.

| Model  | Symbol<br>Equivariance | Sudoku<br>FSR                        | Maze<br>FSR                          |
|--------|------------------------|--------------------------------------|--------------------------------------|
| TRM    | False                  | 71.9 <sup>(71.80–72.08)</sup>        | 85.3 <sup>(82.92–87.41)</sup>        |
| HRM    | False                  | 63.5 <sup>(63.38–63.67)</sup>        | 74.5 <sup>(71.66–77.15)</sup>        |
| SE-RRM | True                   | <b>93.7</b> <sup>(93.66–93.81)</sup> | 0 <sup>(0.00–0.38)</sup>             |
| SE-RRM | False                  | 89.7 <sup>(89.61–89.79)</sup>        | <b>88.8</b> <sup>(86.64–90.65)</sup> |